

Chapter 1

Exact Methods for Multi-objective Combinatorial Optimisation

Matthias Ehrgott, Xavier Gandibleux, and Anthony Przybylski

Abstract In this chapter we consider multi-objective optimisation problems with a combinatorial structure. Such problems have a discrete feasible set and can be formulated as integer (usually binary) optimisation problems with multiple (integer valued) objectives. We focus on a review of exact methods to solve such problems. First, we provide definitions of the most important classes of solutions and explore properties of such problems and their solution sets. Then we discuss the most common approaches to solve multi-objective combinatorial optimisation problems. These approaches include extensions of single objective algorithms, scalarisation methods, the two-phase method and multi-objective branch and bound. For each of the approaches we provide references to specific algorithms found in the literature. We end the chapter with a description of some other algorithmic approaches for MOCO problems and conclusions suggesting directions for future research.

Key words: Multi-objective optimisation; combinatorial optimisation; exact methods; scalarisation; branch and bound; two-phase method

Matthias Ehrgott

Department of Management Science, Lancaster University, Bailrigg, Lancaster LA1 4YX, United Kingdom, e-mail: m.ehrgott@lancaster.ac.uk

Xavier Gandibleux

Faculté des Sciences et des Techniques, Université de Nantes, 44322 Nantes Cedex 03, France, e-mail: Xavier.Gandibleux@univ-nantes.fr

Anthony Przybylski

Faculté des Sciences et des Techniques, Université de Nantes, 44322 Nantes Cedex 03, France, e-mail: a.przybylski@univ-nantes.fr

1.1 Introduction

In this section we provide basic definitions and notations for multi-objective combinatorial optimisation (MOCO) problems, including definitions of efficient solutions and non-dominated points. We discuss the theoretical background of multi-objective combinatorial optimisation. We recall results from computational complexity, highlighting that MOCO problems are almost always NP-hard and #P-hard. Moreover, for many problems, instances with an exponential number of non-dominated points exist. We mention results on the (non)-connectedness of efficient solutions. We also give an outlook on the rest of the chapter.

1.1.1 Definitions

Formally, a multi-objective combinatorial optimisation problem can be written as a linear integer programme with multiple objectives

$$\min\{z(x) = Cx : Ax = b, x \in \{0, 1\}^n\}, \quad (1.1)$$

where $x \in \{0, 1\}^n$ is a (column) vector of n binary variables $x_j, j = 1, \dots, n$; $C \in \mathbb{Z}^{p \times n}$ contains the rows c^k of coefficients of p linear objective functions $z_k(x) = c^k x, k = 1, \dots, p$ and $A \in \mathbb{Z}^{m \times n}, b \in \mathbb{Z}^m$ describe m constraints $a^i x = b_i, i = 1, \dots, m$, where $a^i, i = 1, \dots, m$ are m row vectors from $\mathbb{Z}^n$. The constraints define combinatorial structures such as paths, trees, or cycles in a network or partitions of a set, etc. and it will be convenient to assume that all coefficients, i.e. all entries of A, b , and C , are integers.

In order to define what solving MOCO problem (1.1) means, we define orders of vectors in $\mathbb{R}^p$. We use the notations $\leq, \leq$, and $<$ to define componentwise orders in Definition 1.

Definition 1. Let $y^1, y^2 \in \mathbb{R}^p$. We write

- $y^1 \leq y^2$ if $y_k^1 \leq y_k^2$ for $k = 1, \dots, p$;
- $y^1 \leq y^2$ if $y^1 \leq y^2$ but $y^1 \neq y^2$ and
- $y^1 < y^2$ if $y_k^1 < y_k^2$ for $k = 1, \dots, p$.

According to the componentwise orders, we define $\mathbb{R}_{\geq}^p := \{y \in \mathbb{R}^p : y \geq 0\}$ as the non-negative orthant in $\mathbb{R}^p$, and analogously $\mathbb{R}_{\leq}^p$ and $\mathbb{R}_{>}^p$.

The set $X = \{x \in \{0, 1\}^n : Ax = b\}$ is called feasible set in decision space $\mathbb{R}^n$ and $Y := z(X) = \{Cx : x \in X\}$ is the feasible set in objective space $\mathbb{R}^p$. The set of points $\text{conv}(Y) + \mathbb{R}_{\leq}^p$, sometimes called the Edgeworth-Pareto hull of Y , is very important. Figure 1.1 illustrates the feasible set Y of a MOCO problem with two objectives as a finite set of circles and its Edgeworth-Pareto hull $\text{conv}(Y) + \mathbb{R}_{\leq}^p$ as a shaded area.

Individual minimisers of the p objective functions $z_k(x)$, for $k = 1, \dots, p$, i.e. feasible solutions $\hat{x} \in X$ such that $z_k(\hat{x}) \leq z_k(x)$ for all $x \in X$ for some $k \in \{1, \dots, p\}$

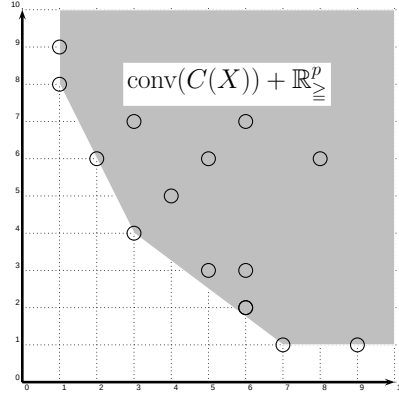


Fig. 1.1 Feasible set and Edgeworth-Pareto hull

do only minimise a single objective function and do not provide any control over the values of the other $p - 1$ objectives if they are not unique, see Figure 1.2a. In that case their objective function vectors may differ in values $z_i(\hat{x})$ for $i \neq k$. It is clear (see Figure 1.2a) that a point combining the minimal values for all objectives, called the ideal point and denoted y^I , i.e. $(1, 1)^T$ in Figure 1.2a, does usually not belong to Y or even $\text{conv}(Y) + \mathbb{R}_{\geq}^p$. Hence an ideal feasible solution $x^I \in X$ such that $z_k(x^I) \leq z_k(x)$ for all $x \in X$ and all $k = 1, \dots, p$ does in general not exist.

The ambiguity in considering individual optima is avoided by turning to lexicographic optima, i.e. feasible solutions $\hat{x}$ such that $z(\hat{x}) \leq_{\text{lex}} z(x)$ for all $x \in X$, and more generally, $z^\pi(\hat{x}) \leq_{\text{lex}} z^\pi(x)$ for all $x \in X$ and some permutation $z^\pi = (z_{\pi(1)}, \dots, z_{\pi(p)})$ of the components of the vector valued function $z = (z_1, \dots, z_p)$. The lexicographic order is defined in Definition 2.

Definition 2. Let $y^1, y^2 \in \mathbb{R}^p$. We write $z^\pi(\hat{x}) <_{\text{lex}} z^\pi(x)$ (y^1 is lexicographically smaller than y^2 with respect to permutation π) if there is some $k \in \{1, \dots, p\}$ such that $y^1_{\pi(i)} = y^2_{\pi(i)}$ for $i = 1, \dots, k-1$ and $y^1_{\pi(k)} < y^2_{\pi(k)}$. Hence $z(\hat{x}) \leq_{\text{lex}} z(x)$ if either $z^\pi(\hat{x}) <_{\text{lex}} z^\pi(x)$ or $z(\hat{x}) = z(x)$.

Figure 1.2a shows non-unique individual minima for z_1 and z_2 (filled circles). Among those, $\hat{y}^1 = (1, 8)^T$ and $\hat{y}^2 = (7, 1)^T$ are lexicographically minimal for the permutation of objectives (z_1, z_2) and (z_2, z_1) , respectively.

While individual and lexicographic optima refer to total (pre)orders on $\mathbb{R}^p$, multi-objective optimisation is based on the concept of efficiency or Pareto optimality. It is defined using partial (pre)orders based on the componentwise comparison of vectors in Definition 1.

Definition 3. A feasible solution $\hat{x} \in X$ belongs to the set of weakly efficient solutions X_{wE} if there is no $x \in X$ with $z(x) < z(\hat{x})$. In that case, $z(\hat{x})$ is called weakly non-dominated. We denote $Y_{wN} := z(X_{wN})$ the set of all weakly non-dominated

points. Feasible solution $\hat{x} \in X_E$, the set of efficient solutions, if there is no feasible x with $z(x) \leq z(\hat{x})$. The objective vector $z(\hat{x})$ of an efficient solution is called non-dominated point and $Y_N := z(X_E)$ is the set of all non-dominated points.

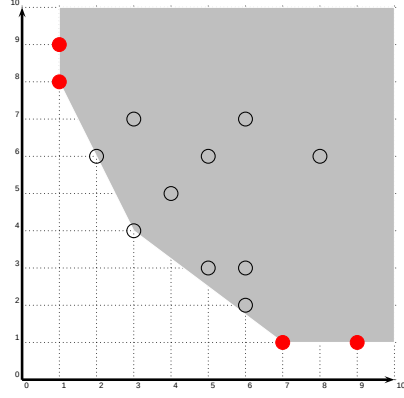


Fig. 1.2a Individual and lexicographic minima

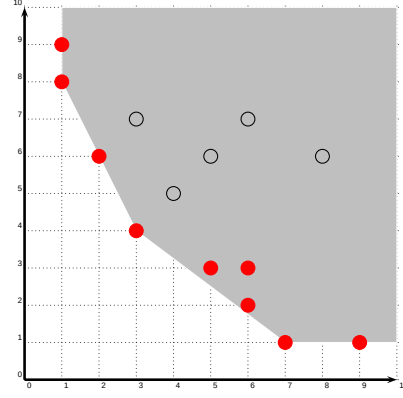


Fig. 1.2b (Weakly) non-dominated points

Definition 4 provides two further definitions of efficiency, that only play a limited role in multi-objective combinatorial optimisation.

Definition 4. Feasible solution $\hat{x} \in X$ is called strictly efficient if there is no $x \in X$ such that $z(x) \leq z(\hat{x})$. It is called properly efficient if it is efficient and there exists a real number $M > 0$ such that for every $i \in \{1, \dots, p\}$ and $x \in X$ such that $z_i(x) < z_i(\hat{x})$ there exists some $j \in \{1, \dots, p\} \setminus \{i\}$ such that $z_j(\hat{x}) < z_j(x)$ and $(z_i(\hat{x}) - z_i(x)) / (z_j(x) - z_j(\hat{x})) \leq M$.

Strict efficiency of $\hat{x}$ implies that the pre-image $z^{-1}(z(\hat{x}))$ is a singleton. In single objective combinatorial optimisation, this corresponds to unique optimisers and is rarely considered. Proper efficiency on the other hand relates to bounded trade-offs between the objectives. While this is an important issue in continuous non-linear multi-objective optimisation (see Chapter 18), in our setting we assume that $Y \subset \mathbb{Z}^p$ and differences between objective values are therefore integers, too, implying that all efficient solutions are properly efficient. The following distinction between supported and non-supported efficient solutions is, on the other hand, crucial in multi-objective combinatorial optimisation.

Definition 5. An efficient solution is supported if there is some $\lambda \in \mathbb{R}_{\geq}^p$ such that $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$. In case $C\hat{x}$ is an extreme point of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$ it is an extreme efficient solution and $C\hat{x}$ is an extreme non-dominated point. We let X_{SE1} be the set of extreme efficient solutions and let X_{SE2} denote the set of supported efficient solutions such that $C\hat{x}$ is in the relative interior of a face of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$. All supported efficient solutions are $X_{SE} = X_{SE1} \cup X_{SE2}$. Finally, $X_{NE} = X_E \setminus X_{SE}$.

is the set of non-supported efficient solutions, i.e. $\hat{x}$ such that $C\hat{x}$ is in interior of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$. The counterparts of X_{SE} and X_{NE} in objective space are called the sets of (non-)supported non-dominated points Y_{SN} and Y_{NN} , respectively. Analogously, Y_{SN1} and Y_{SN2} denote the sets of non-dominated extreme points of Y and supported non-dominated points that are not extreme points of Y , respectively.

It is very easy to construct small examples that show that even the bi-objective shortest path, spanning tree, and assignment problems have non-supported efficient solutions. In our small example, Figure 1.2b shows (weakly) non-dominated points as filled circles. Note that $(1, 9)^T$, $(6, 3)^T$ and $(9, 1)^T$ are weakly non-dominated but not non-dominated. Moreover, $(5, 3)^T$ and $(6, 2)^T$ are non-supported non-dominated points. Figures 1.3a and 1.3b illustrate supported non-dominated points. Figure 1.3a shows that $(3, 4)^T$ is an extreme non-dominated point and Figure 1.3b that $(2, 6)^T$ is a supported non-dominated point in the relative interior of a face of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$.

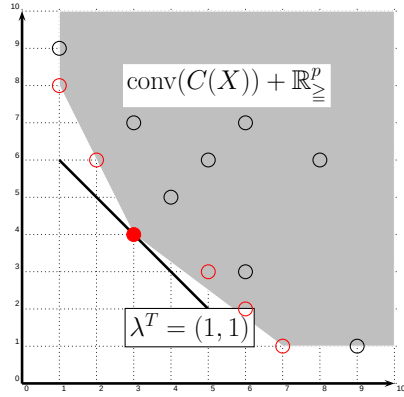


Fig. 1.3a Extreme non-dominated point for $\lambda^T = (1, 1)$

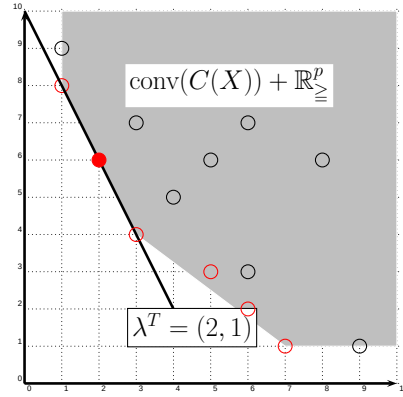


Fig. 1.3b Supported non-dominated point in the relative interior of a face for $\lambda^T = (2, 1)$

Following [40] we call $x^1, x^2 \in X_E$ equivalent if $Cx^1 = Cx^2$. A complete set of efficient solution is a set $\hat{X} \subset X_E$ such that for all $y \in Y_N$ there is some $x \in \hat{X}$ with $z(x) = y$. A minimal complete set contains no equivalent solutions, the maximal complete set X_E contains all equivalent solutions. We can now speak about, e.g. a minimal complete set of extreme efficient solutions. This classification allows to precisely describe what is to be understood by statements that some algorithm “solves” a certain MOCO problem. We shall always understand solving a MOCO problem as finding all non-dominated points and for each $y \in Y_{ND}$ one x such that $Cx = y$, i.e. finding a minimal complete set of efficient solutions.

1.1.2 Computational Complexity

The most striking feature concerning computational complexity in the sense of worst case performance of algorithms is that MOCO problems are hard in the conventional notions of computational complexity. We formally define the decision problem related to optimisation problem (1.1) as: “Given $d \in \mathbb{Z}^p$, does there exist $x \in X$ such that $Cx \leq d$?” and the associated counting problem as: “Given $d \in \mathbb{Z}^p$, how many $x \in X$ satisfy $Cx \leq d$?” Apart from these questions we are also interested in knowing how many efficient solutions (non-dominated points) may exist in the worst case. General upper bounds on the cardinality of Y_N (for MOCO problems in the presence of both sum and bottleneck objectives) are given by [92] and earlier, lower bounds on $|Y_N|$ have been derived in [33].

Obviously, the multi-objective version of any NP-hard single objective combinatorial optimisation problem is also NP-hard. Another source of NP-hardness of MOCO problem (1.1) derives from the NP-hardness of so-called resource-constrained single objective combinatorial optimisation problems. These are problems that ask for the minimisation of a single linear function c^1x over feasible set X subject to the additional constraint $c^2x \leq d$. Since the decision and counting problems associated with a resource-constrained combinatorial optimisation problem are identical to those of bi-objective combinatorial optimisation problems, hardness results for the resource constrained problems imply hardness results for the bi-objective problems.

It is well known that if A is a totally unimodular matrix, then the polyhedron $\text{conv}(X)$ has only integer extreme points. Hence, in the single objective case, (1.1) can be solved by linear programming. The presence of non-supported non-dominated points makes the total unimodularity property much less useful in the multi-objective case. However, [50] show for the bi-objective case, that if the variables of (1.1) are separable, i.e. $x = (x^1, x^2)$ such that $c^kx = c^kx^k$ for $k = 1, 2$ and A is totally unimodular, then the set of non-supported efficient solutions is empty. Hence, in this case, (1.1) can be solved by (parametric) linear programming methods.

Specifically addressing MOCO problems, we first consider a binary optimisation problem without any constraints. [26] uses a parsimonious transformation to the binary knapsack problem, which is known to be NP-complete and #P-complete [44, 103], to show that the most basic MOCO problem, called the unconstrained MOCO problem, is hard.

Theorem 1. *The unconstrained bi-objective combinatorial optimisation problem*

$$\min \left\{ \sum_{i=1}^n c_i^k x_i \text{ for } k = 1, 2 : x_i \in \{0, 1\} \text{ for } i = 1, \dots, n \right\} \quad (1.2)$$

is NP-hard and #P-hard. Moreover, there is an instance of (1.2) that has an exponential number of non-dominated points.

For the latter part of this theorem, it is sufficient to set $c_i^k := (-1)^k 2^{i-1}$ to see that $Y = Y_N$, because the cost coefficients are simply binary representations of the

positive and negative values of integers $1, \dots, 2^{n-1}$. This is a common feature of many of the instances constructed to show that the number of non-dominated points can be exponential in the worst case. As a consequence, all the non-dominated points of such instances lie on a straight line in objective space defined by a constant sum of both objectives. The more interesting question of whether the number of extreme non-dominated points, i.e. $|Y_{SN1}|$, can be exponential in the size of an instance has not been investigated widely. The existence of instances with an exponential number of non-dominated point is called intractability of the MOCO problem.

Although intractability results imply that even Y_{SN} can be exponential in the size of a problem instance, numerical tests reveal that the number of non-dominated points is often “small”, in particular in real world applications. This has been observed by [77] and [62] for randomly generated and real world instances of the shortest path problem and by [73] for randomly generated instances of the bi-objective assignment problem. This evidence suggests that the numerical values of objective function coefficients in C play an important role just as does the combinatorial structure of the instance.

We summarise other complexity results in Table 1.1.

Table 1.1 Complexity results for MOCO problems

MOCO problem	Result	Reference
Bi-objective shortest path	NP-hard	[90]
	$ Y_{SN} $ is exponential	[40]
Bi-objective integer minimum cost flow	$ Y_{SN1} $ is exponential	[82]
Bi-objective minimum spanning tree	NP-hard	[11]
	$ Y_{SN} $ is exponential	[39]
	$ Y_{SN1} = O(\mathcal{E} ^2)$	[84]
	$ Y_{NN} $ is exponential	[83]
Bi-objective global minimum cut	$ Y_N = O(\mathcal{V} ^7)$	[1]
Bi-objective assignment	NP-hard	[90]
	#P-hard	[63]
Bi-objective search problem on a line	NP-hard	[68]
	$ Y_N $ is exponential	[68]
Bi-objective uniform matroid	NP-complete	[24]

Despite these mainly negative results, [8] have shown that encoding the entire set of non-dominated solutions of a multi-objective integer programming problem in a short sum of rational functions is polynomially doable, when the dimension of the decision space is fixed.

1.1.3 Connectedness of Efficient Solutions

One approach to design algorithms to solve MOCO problems is the idea of local search. In local search, a neighbourhood of a feasible solution is defined, i.e. for $x \in X$, $N(x)$ is a subset of X , so that each solution in $N(x)$ can be obtained from x by a certain “move”. Such a move can be, e.g. the exchange of an edge of a spanning tree by another edge of the underlying graph or the basis exchange of a pivot operation in the simplex algorithm. It is then possible to define a graph $\mathcal{EG} = (\mathcal{V}, \mathcal{E})$, called the efficiency graph in [29] and the adjacency graph in [83], the vertices of which are the efficient solutions of a MOCO problem and the edges of which are defined by $[x^1, x^2] \in E$ if and only if $x^2 \in N(x^1)$ and $x^1 \in N(x^2)$. If the adjacency graph of an instance of a MOCO problem is connected, it is then possible to find all efficient solutions of the MOCO problem by a local search procedure. The set X_E is called connected, if the corresponding efficiency graph is connected. [29] show the first result concerning the (non)connectedness of the set of efficient solutions of a MOCO problem.

Theorem 2. *The adjacency graph of the set X_E of an instance of the bi-objective shortest path and bi-objective minimum spanning tree problem is not connected in general.*

[83] notes that this result also holds for the adjacency graph of weakly efficient solutions X_{wE} . Furthermore, [83] demonstrates that the number and cardinality of the connected components of the adjacency graph can be exponential. [71] modify the example used in [29] to show that the efficiency graph of the bi-objective integer minimum cost flow problem need not be connected. [83] also proves non-connectedness results for the bi-objective binary knapsack problem with uniform weights and cardinality constraint, the binary multiple choice knapsack problem with equal weights, the unconstrained MOCO problem (1.2) and the bi-objective assignment problem.

Very few positive results concerning the connectedness of efficient solutions are known. They are generally obtained for very specific instances. For example it is clear that the adjacency graph of efficient solutions of a multi-objective minimum spanning tree problem on a graph that contains exactly one cycle is connected. [38] prove the connectedness of the efficient solutions for a class of two-dimensional knapsack problems with binary weights.

1.1.4 Bounds and Bound Sets

The success of exact algorithms to solve combinatorial optimisation problems often depends on the availability of good lower and upper bounds, i.e. scalars l and u such that $l \leq z^* \leq u$, where z^* is the optimal value of the optimisation problem. Straightforward generalisations of lower and upper bounds are given by the ideal and nadir points y^I and y^N . They are defined in Equations (1.3) and (1.4),

$$y_k^I := \min\{y_k : y \in Y\}; k = 1, \dots, p; \quad (1.3)$$

$$y_k^N := \max\{y_k : y \in Y_N\}; k = 1, \dots, p. \quad (1.4)$$

The ideal and nadir point are tight lower and upper bounds on the values of any non-dominated point, i.e. there is no $y \in Y_N$ that dominates y^I nor that is dominated by y^N , yet for each $k = 1, \dots, p$ there exists some $y \in Y_N$ such that $y_k = y_k^I$ and some $y \in Y_N$ such that $y_k = y_k^N$. However, in general neither the ideal nor the nadir point are feasible and can hence be far away from the non-dominated set. Note that in Figure 1.4a the ideal point $y^I = (1, 1)^T$ and the nadir point $y^N = (7, 8)^T$.

To overcome this drawback it is necessary to combine the notion of a set of non-dominated points with the idea of bounds, leading to the definition of bound sets. A first definition of lower and upper bound sets L and U is given in [100]. A (different) definition for the bi-objective case is due to [27]. Using only part of the conditions of [27], [21] propose a third definition. Here, we formally give the definition of [28]. Note that a set $S \subset \mathbb{R}^p$ is called $\mathbb{R}_{\geq}^p$ -closed, respectively $\mathbb{R}_{\geq}^p$ -bounded if $S + \mathbb{R}_{\geq}^p$ is closed, respectively if there exists some $\hat{s} \in \mathbb{R}^p$ such that $S \subset \hat{s} + \mathbb{R}_{\geq}^p$, see [28].

Definition 6. • A lower bound set L is a $\mathbb{R}_{\geq}^p$ -closed, $\mathbb{R}_{\geq}^p$ -bounded set such that

$$Y_N \subset L + \mathbb{R}_{\geq}^p \text{ and } L \subset \left(L + \mathbb{R}_{\geq}^p\right)_N.$$

- An upper bound set U is a $\mathbb{R}_{\geq}^p$ -closed, $\mathbb{R}_{\geq}^p$ -bounded set such that Y_N is contained in $\text{cl} \left[\left(U + \mathbb{R}_{\geq}^p \right)^c \right]$ and $U \subset \left(U + \mathbb{R}_{\geq}^p \right)_N$, where c indicates the complement of a set in $\mathbb{R}^p$.

Definition 6 implies that neither L nor U contain points dominated by other points from the same set. Moreover, this definition says that the objective vectors of any set of feasible solutions, filtered by dominance, define an upper bound set. In the single objective case the definitions reduce to the usual definitions of lower and upper bound, and in particular encompass the idea of the incumbent best known solution providing an upper bound. Moreover, it preserves the condition that $L = U$ implies $L = U = Y_N$.

An important lower bound set is defined by the non-dominated set of the convex hull of (supported) non-dominated points, $L = (\text{conv}(Y_{SN}))_N$, a set that is sometimes called the non-dominated frontier of MOCO problem (1.1). This is in fact the tightest possible $\mathbb{R}_{\geq}^p$ -convex lower bound set. In Figure 1.4a this will be the union of the three line segments connecting $(1, 8)^T$, $(3, 4)^T$ and $(7, 1)^T$.

Figure 1.4a shows a lower bound set consisting of three line segments that could represent the non-dominated set of the (multi-objective) linear programming relaxation of a MOCO problem (1.1). Figure 1.4b illustrates how six feasible points $U = \{(1, 9)^T, (3, 6)^T, (4, 5)^T, (6, 2)^T, (9, 1)^T\}$ define an upper bound set. In this way, any multi-objective relaxation of MOCO problem (1.1) defines a lower bound set and any set of feasible points (filtered by dominance) defines an upper bound set, preserving this important property from single objective branch and bound. A node of the branch and bound tree can be fathomed whenever its lower bound set is dominated by the upper bound set of the MOCO problem. The arguably simplest and

most often used lower bound set is $L = \{y^I\}$, i.e the set consisting of the ideal point y^I defined as in (1.3). Similarly, an upper bound set can be obtained by the anti-ideal point y^{AI} with $y_k^{AI} := \max\{c^k x : x \in X\}$ or the nadir point y^N (see Equation (1.4)) consisting of the worst objective values over the efficient set. Note that the nadir point is hard to compute for $p \geq 3$, even for linear problems, see [32].

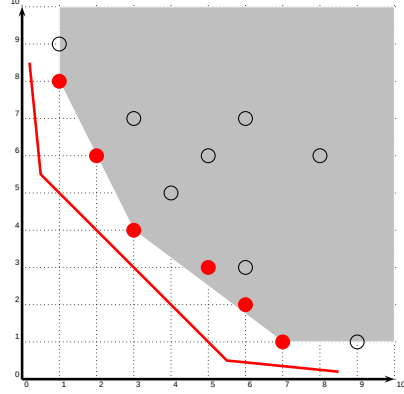


Fig. 1.4a A lower bound set

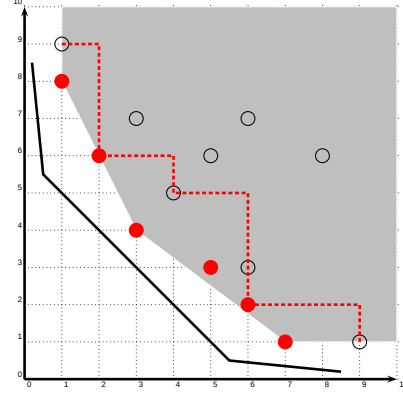


Fig. 1.4b An upper bound set defined by feasible points

1.1.5 Outlook

In what follows, we discuss exact solution approaches proposed in the literature to solve multi-objective versions of combinatorial optimisation problems. This material is presented in four sections. In Section 1.2 we consider algorithms for single objective combinatorial problems that can be extended to solve their multi-objective counterparts. Two prime examples are labelling algorithms for the shortest path problem and the greedy algorithm. In Section 1.3 the topic of scalarisation is discussed, which plays a role in Section 1.4, where we discuss approaches that rely heavily on the repeated solution of single objective combinatorial optimisation problems. Such approaches are particularly beneficial if polynomial time algorithms are available for the single objective problem. We present an outline of the two-phase method [99] and review two-phase algorithms from the literature. More general approaches, that do not revert to repeatedly solve single objective versions of the problem under consideration have been proposed for general MOCO problems. In Section 1.5 we address the principle of branch and bound algorithms and we conclude the survey with a review of other exact solution methods. The chapter concludes with a brief discussion and suggestions for further research.

1.2 Extending Single Objective Algorithms

In this section, we shall discuss algorithms to solve multi-objective combinatorial optimisation problems for which efficient polynomial time algorithms exist to solve their single objective versions, and for which it is possible to extend these algorithms to deal with the multi-objective versions. The prime examples are the multi-objective shortest path and spanning tree problems.

1.2.1 Labelling Algorithms

Let $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ be a directed graph defined by node set $\mathcal{V}$ and arc set $\mathcal{A}$ with p arc costs $c_{ij}^k, k = 1, \dots, p$ on arcs $(i, j) \in \mathcal{A}$. The single-source single-sink multi-objective shortest path problem is to find a minimal complete set of efficient paths from an origin node s to a destination node t . The single-source variant finds a complete set of efficient paths from origin s to all other nodes of $\mathcal{G}$, and the all-pairs version efficient paths between all pairs of origins and destinations.

In the single objective case $p = 1$, label setting algorithms, such as Dijkstra's algorithm [23], or label correcting algorithms, such as Bellman's algorithm [5], are well known polynomial time algorithms.

Multi-objective labelling algorithms rely on the following fact. Assuming that all $c_{ij}^k \geq 0$, let P_{st} be an efficient path from s to t . Then any subpath P_{uv} from u to v , where u and v are nodes on P_{st} , is an efficient path from u to v . Notice that, on the other hand, concatenations of efficient paths need not be efficient. This principle of optimality shows that the multi-objective shortest path problem is an example of multi-objective dynamic programming and implies that generalisations of both Dijkstra's and Bellman's algorithms are possible. Such algorithms have vector valued labels and therefore, due to the partial orders used, need to maintain sets of non-dominated labels at each node rather than single labels.

For a label setting algorithm, lists of permanent and temporary labels are stored and it is necessary to ensure that a permanent label defines an efficient path from s to the labelled node. This can be done by selecting the lexicographically smallest label from the temporary list to become permanent. A label setting algorithm then follows the same steps as in the single objective case, only that newly created labels need to be compared with label sets. New labels dominating existing labels lead to the deletion of those dominated labels. An existing label dominating a new label results in the new label being discarded, otherwise the new label is added to the label list. An interesting feature of the multi-objective shortest path problem is that, in contrast to the single objective one, it is not possible to terminate a multi-objective label setting algorithm once the destination node t has been reached, since further unprocessed temporary labels at nodes in $\mathcal{V} \setminus \{s, t\}$ may lead to the detection of more efficient paths from s to t , another effect of the partial order. Nevertheless, any label at t can serve as an upper bound and temporary labels at any intermediate node dominated by such an upper bound can be eliminated.

Just as in the single objective case, label setting algorithms fail if negative arc lengths are permitted. In the multi-objective case the following situations may occur: If there is a cycle C with $\sum_{(i,j) \in C} c_{ij} \leq 0$ there is no efficient path; if there is a cycle C with $\sum_{(i,j) \in C} c_{ij}^k < 0$ and $\sum_{(i,j) \in C} c_{ij}^l > 0$ for some k and some $l \neq k$ there are infinitely many efficient paths as every pass of the cycle reduces one objective and increases another thereby creating one more efficient path every time. In the presence of negative arc lengths label correcting algorithms are necessary. Once again, one proceeds with processing the labels as in the single objective case, keeping in mind that all newly created labels need to be compared with existing label sets so that all dominated labels can be eliminated. In each iteration either a single label or a set of labels is processed and extended along an arc (arcs) out of the node where the label(s) reside. Of course, no label is permanent until termination of the algorithm. For details on a variety of multi-objective shortest path problems including pseudocode and numerical results the reader is referred to [77]. Another review of multi-objective shortest path algorithms is provided in [96]. [67] report on a computational study of a variety of labelling algorithms for the multi-objective shortest path problem. Newer labelling algorithms can be found for example in [41], [88], [85] and [89].

More generally than algorithms for shortest path problems, dynamic programming can be generalised to multiple objective problems along the same lines as labelling algorithms, see for example [18, 100]. [81] use dynamic programming to solve bi-objective binary knapsack problems.

1.2.2 Greedy Algorithms

Another problem that we discuss is the multi-objective spanning tree problem. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with edge set $\mathcal{E}$ and edge costs $c_{ij}^k, k = 1, \dots, p$ for all edges $[i, j] \in \mathcal{E}$ its aim is to find efficient spanning trees of $\mathcal{G}$, i.e. spanning trees the cost vectors of which are non-dominated. The following theorem is the foundation for greedy algorithms that generalise Prim's [70] and Kruskal's [51] algorithms.

Theorem 3. [39] *Let T be an efficient spanning tree of $\mathcal{G}$. The following assertions hold.*

1. *Let $e \in \mathcal{E}(T)$ be an edge of T . Let $(\mathcal{V}(T_1), \mathcal{E}(T_1))$ and $(\mathcal{V}(T_2), \mathcal{E}(T_2))$ be the two connected components of $\mathcal{G} \setminus \{e\}$. Let $C(e) := \{f = [i, j] \in \mathcal{E} : i \in \mathcal{V}(T_1), j \in \mathcal{V}(T_2)\}$ be the cut defined by deleting e . Then $c(e) \in \min\{c(f) : f \in C(e)\}$.*
2. *Let $f \in \mathcal{E} \setminus \mathcal{E}(T)$ and let $P(f)$ be the unique path in T connecting the end nodes of f . Then $c(f) \leq c(e)$ does not hold for any $e \in P(f)$.*

The first statement of Theorem 3 shows that starting from a single node and adding efficient edges between nodes already included and nodes not yet included in the tree will eventually construct all efficient trees. Notice that several such edges might exist, and hence in every iteration there will be a set of efficient partial trees.

As in the multi-objective shortest path problem adding edges to an efficient partial tree does not necessarily lead to another efficient partial tree, but also adding efficient edges to a dominated partial tree may yield an efficient partial tree. It is therefore necessary to filter out dominated trees at termination of the algorithm. In a similar way, the second statement provides a justification for a Kruskal-like algorithm for the multi-objective spanning tree problem, see [15] for an adaptation to the multi-objective case.

Since spanning trees are examples of matroid bases, the above extends to multi-objective matroid optimisation and the greedy algorithm. A result by [90] shows that for the multi-objective matroid bases can be found by the greedy algorithm working with a topological order of the cost vectors of the elements. Let $\mathcal{M} = (\mathcal{E}, \mathcal{I})$ be a matroid and $c_j \in \mathbb{R}^p$ be the cost vectors of elements $e_j \in \mathcal{E}$. Recall that the componentwise order $\leq$ is a partial order. A topological order of the elements of $\mathcal{E}$ is a total order $\preceq$ such that $c_j \leq c_{j'}$ implies $e_j \preceq e_{j'}$.

Theorem 4. [90] *Let B be an efficient matroid base. Then there exists a topological order of the elements of $\mathcal{E}$ such that the greedy algorithm applied to this order yields B .*

Notice that Theorem 4 provides a necessary, but not a sufficient condition.

Greedy algorithms have also been developed in [37] to solve three-objective unconstrained combinatorial optimisation problems in polynomial time.

1.3 Scalarisation

The idea of scalarisation is to convert a multi-objective optimisation problem to a (parameterised) single objective problem that is solved repeatedly with different parameter values. We are interested in the following desirable properties of scalarisations [104]. Correctness requires that an optimal solution of the scalarised problem is (at least weakly) efficient. A scalarisation method is complete, if all efficient solutions can be found by solving a scalarised problem with appropriately chosen parameters. For computability it is important that the scalarisation is not harder than the single objective version of the MOCO problem. This relates to theoretical computational complexity as well as computation time in practice. Furthermore, since MOCO problem (1.1) has linear constraints and objectives, the scalarisation should have a linear formulation.

The scalarisation techniques that are most often applied in MOCO, illustrated in Figure 1.5a – 1.6, are the weighted sum method (Figure 1.5a)

$$\min \{ \lambda^T z(x) : x \in X \}, \quad (1.5)$$

the ε -constraint method (Figure 1.5b)

$$\min \{ z_l(x) : z_k(x) \leq \varepsilon_k, k \neq l, x \in X \}, \quad (1.6)$$

and the weighted Chebychev method (Figure 1.6)

$$\min \left\{ \max_{k=1,\dots,p} v_k(z_k(x) - y_k^I) : x \in X \right\}. \quad (1.7)$$

In Table 1.2 we summarise the properties of the methods listed in Equations (1.5) – (1.7).

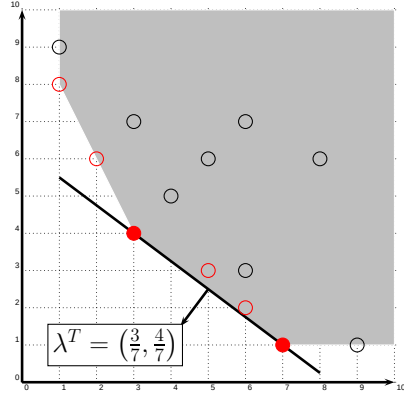


Fig. 1.5a The weighted sum scalarisation

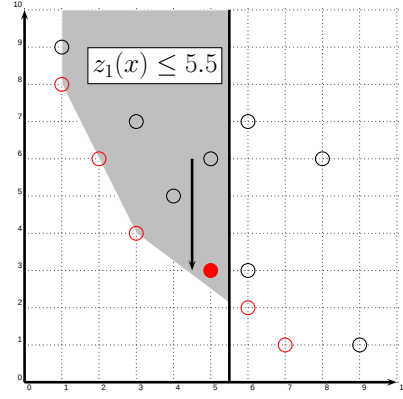


Fig. 1.5b The ε -constraint scalarisation

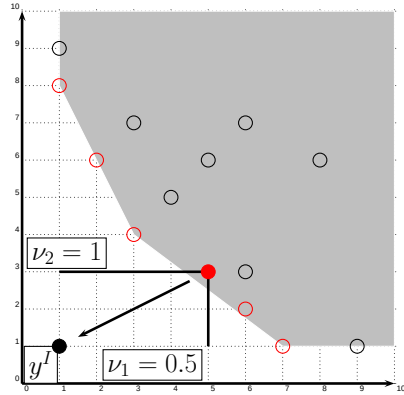


Fig. 1.6 The Chebychev scalarisation

All three scalarised problems clearly have a linear formulation (with integer variables, of course). They do also all compute (weakly) efficient solutions, see, e.g. [26] for proofs. As for completeness, it follows from the definition of non-supported non-dominated points, that the weighted sum scalarisation cannot compute any non-supported efficient solution. On the other hand, both the ε -constraint scalarisation

Table 1.2 Properties of popular scalarisation methods

Scalarisation	Correct	Complete	Computable	Linear
Weighted sum	+	–	+	+
ε -constraint	+	+	–	+
Chebychev	+	+	–	+

and the Chebychev scalarisation can be tuned to find all efficient solutions. For proofs we do again refer the reader to, e.g. [26]. In terms of computability, we have seen before that the weighted sum scalarisation (1.5) does maintain the structure of the single objective variant of MOCO problem (1.1), it is therefore solvable with the same computational effort. On the other hand, the ε -constraint method (1.6) and the (linearised version of the) Chebychev scalarisation (1.7) both include bounds on objective function values. Such additional constraints usually make single objective versions of MOCO problem (1.1) harder to solve, because they destroy the structure of the problem which is exploited in efficient algorithms for their solution.

In [25] it has been shown that all three scalarisations (and several others) are special cases of the more general formulation

$$\min_{x \in X} \left\{ \max_{k=1}^p [v_k(c_k x - \rho_k)] + \sum_{k=1}^p [\lambda_k(c_k x - \rho_k)] : c_k x \leq \varepsilon_k, k = 1, \dots, p \right\}, \quad (1.8)$$

where v and λ denote (non-negative) weight vectors in $\mathbb{R}^p$, $\rho \in \mathbb{R}^p$ is a reference point and scalars ε_k represent bounds on objective function values. To see this, set $v_k = 0, \rho_k = 0$ and $\varepsilon_k = M$ for all $k = 1, \dots, p$ and sufficiently large M to obtain the weighted sum problem (1.5); $v_k = 0$ for $k = 1, \dots, p$, $\lambda_l = 1, \lambda_k = 0$ for all $k \neq l$, $\varepsilon_l = M$ and $\rho_k = 0$ for all $k = 1, \dots, p$ for the ε -constraint scalarisation (1.6); and finally $\rho = y^I, \lambda = 0, \varepsilon_k = M$ for all $k = 1, \dots, p$ for the Chebychev scalarisation (1.7). With regard to the general scalarisation in Equation (1.8) we cite the following result.

Theorem 5. [25]

1. The general scalarisation (1.8) is correct, complete, and NP-hard.
2. An optimal solution of the Lagrangean dual of the linearised general scalarisation is a supported efficient solution of the MOCO problem (1.1).

Theorem 5 shows that the general scalarisation will be difficult to solve and that solving it by Lagrangian relaxation is not useful to obtain non-supported non-dominated points. Moreover, Table 1.2 indicates that complete scalarisations are not computable, whereas computable ones are not complete. To resolve this dilemma, one may ask whether it is possible to come up with a compromise, that is a scalarisation that falls somewhere between the weighted sum scalarisation and the ε -constraint/Chebychev scalarisation, combining their strengths and eliminating their weaknesses. This is indeed possible with the elastic constraint scalarisation. This

scalarisation is derived from the ε -constraint scalarisation, but allows the constraints on objective function values to be violated, with the violation penalised in the objective function. Formally, the elastic constraint scalarisation is defined as in Equation (1.9),

$$\min \left\{ c_l x + \sum_{k \neq l} \mu_k w_k : c_k x + v_k - w_k = \varepsilon_k, k \neq l; \quad x \in X; \quad v_k, w_k \geq 0, k \neq l \right\}. \quad (1.9)$$

The constraints on objective values in the ε -constraint scalarisation are turned into equality constraints by means of slack and surplus variables v_k and w_k . Positive values of w_k indicate constraint violations in the ε -constraint scalarisation and are penalised with a contribution of penalty parameter μ_k in the objective function. Figure 1.7a and 1.7b compare these two scalarisations. Figures 1.7a repeats Figure 1.5b, where the vertical line indicates a hard constraint on $z_1(x) \leq 5.5$ and the arrow indicates minimisation of z_2 . The optimal point is indicated by a filled circle. Figure 1.7b shows that points to the right of the vertical line are feasible in the elastic constraint scalarisation, with lighter shading indicating greater violation of the limit of ε on z_2 . To the right of the vertical line, for every feasible point of the MOCO problem, an arrow indicates the objective value of the same point in Equation (1.9). Note that the optimal point for Equation (1.9) in this example is now to the right of the vertical line. Theorem 6 summarises the properties of the elastic constraint scalarisation.

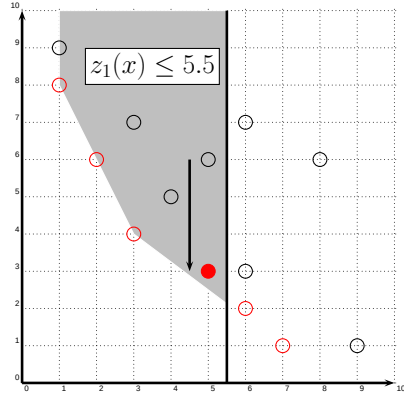


Fig. 1.7a The ε -constraint scalarisation

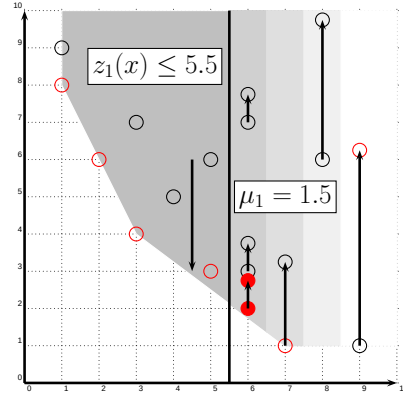


Fig. 1.7b The elastic constraint scalarisation

Theorem 6. [31] *The method of elastic constraints is correct and complete. It contains the weighted sum and ε -constraint method as special cases.*

For a proof of the first part we refer to [31]. The second part follows by setting first $\varepsilon_k < \min_{x \in X} c^k x$ for $k \neq l$. Then $v_k = 0$ for all feasible solutions $x \in X$ and hence

$w_k = c^k x - \varepsilon_k$ and the problem reduces to the weighted sum problem. Secondly, we can set $\mu_k = M$ for a sufficiently large number M . Then any feasible point with $y_k > \varepsilon_k$ will contribute so much penalty to the objective function of elastic constraint scalarisation (1.9) that it is not optimal. Hence scalarisations (1.9) and (1.6) will have the same set of optimal solutions.

Although the elastic constraint scalarisation (1.9) is *NP*-hard, it is often solvable in reasonable time practice because it respects problem structure better than the ε -constraints of Equation (1.6). It limits the damage done by adding hard ε -constraints to the model. A successful implementation of the method for an application to bi-objective set partitioning problems has been reported in [30].

When solving a MOCO problem using scalarisation, bearing in mind that the scalarised combinatorial optimisation problems may be hard to solve, it is of interest to know how many scalarised problems need to be solved in order to find a minimal complete set of efficient solutions. Theorem 7 summarises the results in this regard.

Theorem 7. *1. In the case $p = 2$, the number of scalarised single objective problems to be solved in order to determine Y_N is bounded by $2|Y_N| - 1$ [12, 79]. In case the ε -constrained scalarisation is used, this bound is $|Y_N| + 1$ [53].
2. In the case $p = 3$ the bound is $3|Y_N| - 2$ and $2|Y_N| - 1$ for the ε -constraint scalarisation [17]. For $p > 3$ the general bound is $O(|Y_N|^{\lfloor \frac{p}{2} \rfloor})$, [48]*

We note that general scalarisation algorithms to solve MOCO problems often make use of scalarisations that use resource constrained single objective problems of the form

$$\min\{g(f(x)) : f(x) \leq u, x \in X\}, \quad (1.10)$$

where $g : \mathbb{R} \rightarrow \mathbb{R}$ is a strongly increasing function, see [48] for a general scheme and the references in Section 1.3.1 for specific examples of such algorithms. These algorithms work with an upper bound set that is updated throughout the algorithm. They end as soon as it is guaranteed that all non-dominated points have been found.

1.3.1 Scalarisation Algorithms from the Literature

In this section we present a tabular overview of scalarisation algorithms for MOCO problems. Most of these algorithms are generic, in the sense that they can be applied to any multi-objective integer programming problem. Hence Table 1.3 presents the type of scalarisation applied, for which MOCO problem the algorithm has been tested, and the references. If the algorithm has been specifically developed for a particular MOCO problem, this is denoted by (*) appearing behind the problem.

Table 1.3 Algorithms based on scalarisation

MOCO problem	Scalarisation	Reference
BO binary LP	Weighted sum with ε -constraints	[12]
BO knapsack, capacitated network routing	Weighted Chebychev	[79]
TO multidimensional knapsack	Lexicographic ε -constraint	[53]
Generic	Weighted sum with ε -constraints	[48]
TO three-dimensional knapsack	General scalarisation	[17]
BO integer minimum cost flow (*)	ε -constraint	[34]
BO knapsack	Lexicographic weighted Chebychev	[86]
BO multidimensional knapsack	Weighted sum with constraints	[94]
TO three-dimensional knapsack	Lexicographic ε -constraint	[64]
MO knapsack, shortest path, spanning tree	Lexicographic ε -constraint	[55]
MO three-dimensional knapsack, assignment	Lexicographic ε -constraint	[65]
MO TSP		
TO knapsack, assignment	Lexicographic ε -constraint	[46]
BO knapsack	Augmented weighted Chebychev	[16]
MO integer LP	Single objective with constraints	[49]
BO, TO multidimensional knapsack	Augmented ε -constraint	[59]
BO shortest path		
BO set partitioning (*)	Elastic constraint	[30], [95]
BO TSP with profits (*)	ε -constraint	[6]

1.4 The Two-Phase Method

If it is not possible to adapt a single objective algorithm to the multi-objective case directly, then it is desirable to use the single objective algorithm to solve single objective instances of the multi-objective problem (repeatedly) to obtain some efficient solutions. This is in particular true for problems for which the single objective counterpart is solvable in polynomial time. In this section we consider algorithms for such multi-objective optimisation problems. Considering the hardness of even the “simplest” bi-objective combinatorial optimisation problems as discussed in Section 1.1.1, this indicates that there is benefit in solving the single objective version as a subproblem even if that has to be done often, so that the polynomial time algorithms can be exploited as much as possible. We shall describe the two phase method, originally proposed by [99], as a general algorithmic framework to solve multi-objective combinatorial optimisation problems in a way that relies heavily on the use of fast algorithms for single objective optimisation. A detailed exposition of the method can be found in [76].

1.4.1 The Two Phase Method for Two Objectives

We first explain the two phase method for MOCO problems with two objectives. In Phase 1 at least a (minimal) complete set of extreme efficient solutions X_{SE1} is

found. This is typically done starting from two lexicographically optimal solutions. The dichotomic method then recursively calculates a weight vector $\lambda^T = (\lambda_1, \lambda_2) \in \mathbb{R}_{>}^2$ as a normal to the line connecting two currently known non-dominated points y^l and y^r with $y_1^l < y_1^r$ (and therefore $y_2^l > y_2^r$ according to equation (1.11) and solving a weighted sum problem $\min\{\lambda^T Cx : x \in X\}$, where

$$\lambda := (y_2^l - y_2^r, y_1^r - y_1^l). \quad (1.11)$$

In Figure 1.8a, lexicographic minima $(1, 8)^T$ and $(7, 1)^T$ are identified and define $\lambda^T = (8 - 1, 7 - 1) = (7, 6)$. The corresponding weighted sum problem yields non-dominated point $(3, 4)^T$ (Figure 1.8b) and allows dividing the problem in two: The first weighted sum problem with $\lambda^T = (8 - 4, 3 - 1) = (4, 2)$ looks for new non-dominated extreme points between $(1, 8)^T$ and $(3, 4)^T$. The second with $\lambda^T = (4 - 1, 7 - 3) = (3, 4)$ is used to explore the area between $(3, 4)^T$ and $(7, 1)^T$. Neither finds any further non-dominated extreme points in the example. The first, however, may or may not find supported (but non-extreme) non-dominated point $(2, 6)^T$, depending on the solver (Figure 1.9).

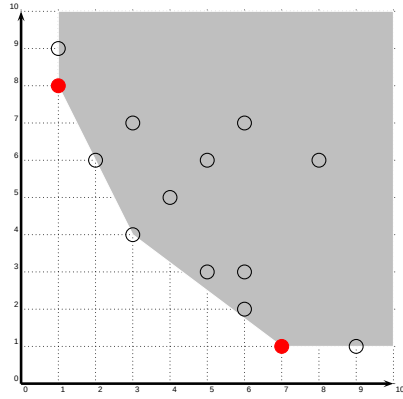


Fig. 1.8a Lexicographically optimal points

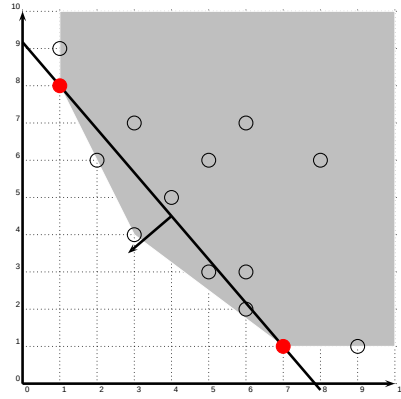


Fig. 1.8b The first weighted sum problem

This dichotomic scheme has been independently described by [14], [2] and [22]. Alternatively, it is possible to apply a parametric scheme, which starts with one of the lexicographically optimal points, say for permutation (z^1, z^2) of the objective functions, and then moves to further extreme non-dominated points by systematically increasing the weight of the second objective function in the weighted sum problem. This scheme is usually applied, if the single objective version of MOCO problem (1.1) can be solved by linear programming. In this case a parametric version of the simplex algorithm to solve bi-objective linear programmes is applied, see, e.g. [77] and references therein for further details. Since we assume that all objective function values are integer, lexicographically optimal solutions can be found by solving two single objective problems, as [73] have observed. First, weighted sum problem $\min\{\lambda^T Cx : x \in X\}$ is solved with $\lambda^T = (1, 0)$. Since an optimal so-

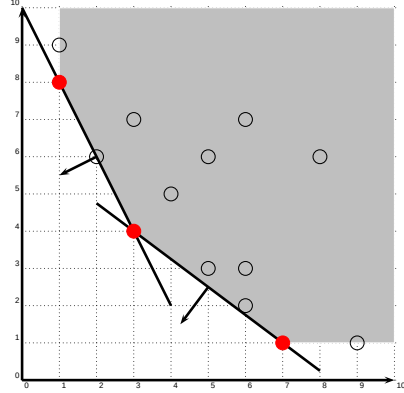


Fig. 1.9 Phase 1 of the two phase method

lution of this problem may be weakly efficient, resulting in weakly non-dominated point y^1 the weighted sum problem is then resolved with $\lambda^T = (y_2^1 + 1, 1)$ to either confirm that y^1 is the lexicographically optimal point or to replace it by the true lexicographically optimal point.

In Phase 2 any other non-dominated points are determined, in particular non-supported non-dominated points Y_{NN} . After Phase 1 it is possible to restrict the search for non-dominated points to triangles defined by consecutive non-dominated extreme points, see Figure 1.10a. In the literature, several methods have been proposed to achieve that. Neighbourhood search as suggested, e.g. in [54, 87] is in general wrong, because efficient solutions are in general not connected via a neighbourhood structure, see Section 1.1.3. One can apply constraints to restrict objective values to triangles and modify those constraints as further points are discovered. However, the computational effort may be high, in particular because problems tend to get harder when adding further (knapsack type) constraints, see Section 1.1.2. This idea is therefore contrary to the spirit of the two phase method. Variable fixing strategies as suggested by [99] can be a reasonable alternative. However, currently the best performing two phase algorithms are those that exploit a ranking algorithm that generates solutions of single objective (weighted sum) problems in order of their objective value (Figure 1.10b). The ranking method has been successfully applied for a number of bi-objective problems, namely bi-objective assignment [73], multi-modal assignment [69], spanning tree [93], shortest path [77], network flow [78], and binary knapsack [42]. It is important to note that for all these problems efficient ranking algorithms to find r -best solutions of the single objective version of the problem are available, references to which can be found in the original papers.

The ranking algorithm is applied to weighted sum problems corresponding to each triangle (see Figure 1.10b). Let $\lambda \in \mathbb{R}_{>}^2$ be a weight vector defined as the normal to the line connecting two consecutive non-dominated extreme points, e.g. $\lambda^T = (4, 2)$ (left triangle) and $\lambda^T = (3, 4)$ (right triangle) in Figure 1.10b. The ranking algorithm then finds second, third, etc. best solutions for the problem of min-

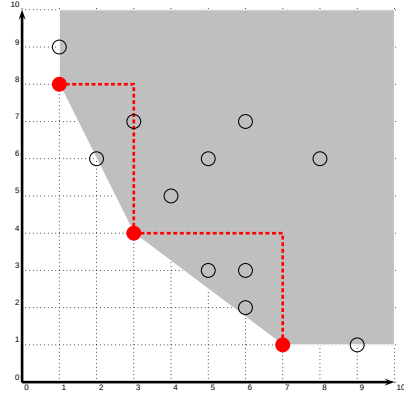


Fig. 1.10a The triangles where non-supported non-dominated points may be located

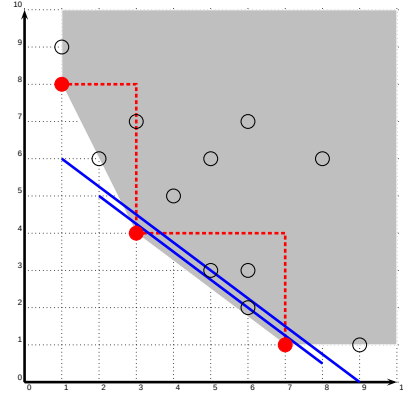


Fig. 1.10b Ranking non-supported non-dominated points

imising $\lambda^T Cx$, in increasing order of objective values for $\lambda^T Cx$ as shown in Figure 1.10b. The two non-dominated extreme points defining the triangle (and λ according to equation (1.11)) define optimal values for this problem.

This process stops at the latest once a solution, the (weighted sum) objective function value of which is worse than that of the third corner of the triangle, is found. In Figure 1.10b, with $\lambda^T = (3, 4)$ the algorithm finds $(6, 2)^T$ with objective 26, then $(5, 3)^T$ with objective value 27. Both points are non-dominated. The 4th best point is $(6, 3)^T$, which is dominated by both previous points. The algorithm can now be stopped, because any further point would also be dominated. This indicates that the enumeration of solutions can be stopped even before the ranking algorithm has identified a feasible solution outside the triangle.

In order to facilitate this, upper bounds on the weighted sum objective value of any efficient solution in the triangle can be derived. Let $\{x^i : 0 \leq i \leq q\}$ be feasible solutions with $z(x^i) \in \Delta(x^0, x^q)$, the triangle defined by $z(x^0)$ and $z(x^q)$, sorted by increasing value of z_1 , where $z(x^0)$ and $z(x^q)$ are two consecutive non-dominated extreme points of $\text{conv}(Y)$. Let

$$\beta_0 := \max_{i=0}^{q-1} \{\lambda_1 z_1(x^{i+1}) + \lambda_2 z_2(x^i)\}, \quad (1.12)$$

$$\beta_1 := \max \left\{ \max_{i=1}^{q-1} \{\lambda^1 z^1(x^i) + \lambda^2 z^2(x^i)\}, \right. \\ \left. \max_{i=0}^q \{\lambda^1 (z^1(x^i) - 1) + \lambda^2 (z^2(x^{i-1}) - 1)\} \right\}, \quad (1.13)$$

$$\beta_2 := \max_{i=0}^q \{\lambda^1 (z^1(x^i) - 1) + \lambda^2 (z^2(x^{i-1}) - 1)\}. \quad (1.14)$$

Then $\beta_0 \geq \beta_1 \geq \beta_2$ are upper bounds for the weighted sum objective value of any non-dominated point in $\Delta(x^0, x^q)$. The ranking process can be stopped as soon as

bound β_j is reached. The use of β_0 and β_1 from Equations (1.12) and (1.13) allows the computation of the maximal complete set, whereas β_2 guarantees a minimal complete set only. Notice that these bounds will be improved every time a new efficient solution is found. In Figure 1.10b, the initial bound β_2 using Equation (1.14) with $\{y^0 = (3, 4)^T, y^1 = (7, 1)^T\}$ is 30, but once $(6, 2)^T$ is found this improves to 27 and with $(5, 3)^T$ also discovered to 24. Clearly no additional feasible point in the triangle can be better than that bound.

The two phase method involves the solution of enumeration problems. In order to find a maximal complete set one must, of course, enumerate all optimal solutions of $\min_{x \in X} \lambda^T Cx$ for all weighted sum problems solved in Phase 1 and enumerate all $x \in X_{NE}$ with $Cx = y \in Y_{ND}$ for all non-supported non-dominated points y . But even in order to compute a minimal complete set enumeration is necessary to find X_{SE2} . There can indeed be many optimal solutions of $\min_{x \in X} \lambda^T Cx$ that are non-extreme and not equivalent to one another (see, e.g. points $(1, 8)^T, (2, 6)^T, (3, 4)^T$ in Figures 1.8a).

As an example of an effective implementation of a two phase method, [73] have developed a two phase algorithm for the bi-objective assignment problem using the Hungarian method [52] to solve weighted sum assignment problems, an enumeration algorithm by [36] to enumerate all optimal solutions of these problems, and a ranking algorithm for (non-optimal) solutions of $\min_{x \in X} \lambda^T Cx$ by [13]. The algorithm outperformed a two phase method using variable fixing, a two phase method using a heuristic to find good feasible solution before Phase 2, general exact MOCO algorithms, and CPLEX using constraints on objectives. An explanation for the good performance of the method is given by the distribution of objective function values for randomly generated instances.

1.4.2 The Two Phase Method for Three Objectives

Extending the two phase method to three objectives is not trivial. In Phase 1, a weight vector defines the normal to a plane. While three non-dominated points are sufficient to calculate such a weight vector, there may be up to six different lexicographically optimal points, so it is unclear which points to choose for calculating weights to start with. Moreover, even if the minimisers of the three objective functions are unique (and therefore there are only three lexicographically optimal solutions), the normal to the plane defined by three non-dominated points may not be positive. In this case no further non-dominated points would be calculated, see an example presented in [74].

Hence, a direct generalisation of the two phase method already fails with the initialisation. Two generalisations of Phase 1 have been proposed by [74] and [66]. We present the ideas of [74], where Phase 1 relies on decomposition of the simplex of all non-negative normalised weights

$$W^0 := \left\{ \lambda \in \mathbb{R}_{\geq}^p : \lambda_p = 1 - \sum_{k=1}^{p-1} \lambda_k \right\} \quad (1.15)$$

into subsets

$$W^0(y) := \{ \lambda \in W^0 : \lambda^T y \leq \lambda^T y' \text{ for all } y' \in Y \} \quad (1.16)$$

of W^0 consisting of all weight vectors λ such that y is a point minimising $\lambda^T y$ over Y . It turns out that y is a non-dominated extreme point if and only if $W^0(y)$ has dimension $p - 1$. This allows us to define adjacency of non-dominated extreme points as follows. Non-dominated extreme points y^1 and y^2 are adjacent if and only if $W^0(y^1) \cap W^0(y^2)$ is a polytope of dimension $p - 2$, which then makes it possible to derive the optimality condition of Theorem 8.

Theorem 8. [74] *If S is a set of supported non-dominated points then*

$$Y_{SN1} \subseteq S \iff W^0 = \bigcup_{y \in S} W^0(y). \quad (1.17)$$

The results above lead to a new Phase 1 algorithm. Let S be a set of supported non-dominated points. Let $W_p^0(y) = \{ \lambda \in W^0 : \lambda^T y \leq \lambda^T y^* \text{ for all } y^* \in S \}$. Then $W^0(y) \subseteq W_p^0(y)$ for all $y \in S$ and $W^0 = \bigcup_{y \in S} W_p^0(y)$. The algorithm initialises S with the lexicographically optimal points. While S is not empty it chooses $\hat{y} \in S$, computes $W_p^0(\hat{y})$ and investigates all facets F of $W_p^0(\hat{y})$ defined by $\lambda^T \hat{y} = \lambda^T y'$ for $y' \in S$ to determine whether F is also a facet of $W^0(\hat{y})$. If $\hat{y}$ minimises $\lambda^T y$ for all $\lambda \in F$ then $\hat{y}$ and y' are adjacent and F is the common face of $W^0(\hat{y})$ and $W^0(y')$. If there are $y^* \in Y$ and $\lambda \in F$ such that $\lambda^T y^* < \lambda^T y$ then $W^0(\hat{y})$ is a proper subset of $W_p^0(\hat{y})$, y^* is added to S and $W_p^0(\hat{y})$ is updated.

At the end of Phase 1, all non-dominated extreme points of Y and a complete set of extreme efficient solutions is known. Any other supported efficient solutions must be optimal solutions to weighted sum problems with λ belonging to 0- and 1-dimensional faces of some $W^0(y)$ of some non-dominated extreme point y . To find these, for each $y \in Y_{SN1}$, we find all optimal solutions of weighted sum problems (1.5) defined firstly by weight vectors λ that are extreme points of $W^0(y)$ which are not located on the boundary of W^0 , and secondly by weight vectors λ located in the interior of edges of $W^0(y)$ the extreme points of which belong to the boundary of W^0 .

To find non-supported non-dominated points one can once again employ a ranking algorithm for weighted sum problems, where λ is chosen to be a normal to a facet defining hyperplane of $\text{conv}(Y) + \mathbb{R}_{\leq}^p$. Note that this is analogous to the bi-objective case. The difficult part in the completion of the Phase 2 algorithm is the computation of good upper bounds and the selection of weights in a way that keeps the ranking of solutions as limited as possible. Here the difficulty arises from the fact that unlike in the case of two objectives, the area where non-supported non-dominated points can be found does not decompose into disjoint subsets, see also Section 1.1.4. Details about Phase 2 for multi-objective combinatorial optimisation

problems can be found in [75], where its generalisation to more than three objectives via a recursive scheme is also discussed.

Numerical results in [72] show that the method outperforms three general methods to solve MOCO problems by a factor of up to 1000 on the three objective assignment problem. The biggest advantage of the two phase method is that it respects problem structure, thereby enabling to use efficient algorithms for single objective problems as much as possible.

1.4.3 Two-Phase Algorithms from the Literature

The two-phase algorithms from the literature are summarised in Table 1.4. We list the MOCO problem for which the algorithm has been designed, the method used for Phases 1 and 2, and the reference.

Table 1.4 Two-phase Algorithms

MOCO problem	Phase 1 Approach	Phase 2 Approach	Reference
Bi-objective integer network flow	Parametric	Local search	[54]
Bi-objective integer network flow	Parametric	Local search	[87]
Bi-objective integer network flow	Parametric	Ranking	[78]
Bi-objective assignment	Dichotomic	Variable fixing	[99]
Bi-objective assignment	Dichotomic	Variable fixing	[97]
Bi-objective assignment	Dichotomic	Ranking	[73]
Three-objective assignment	Dichotomic	Ranking	[74, 75]
Bi-objective multimodal assignment	Dichotomic	Ranking	[69]
Bi-objective spanning tree	Dichotomic	Ranking, branch and bound	[93]
Bi-objective shortest path	Parametric	Label correcting	[61]
Bi-objective shortest path	Dichotomic	Label correcting, label setting	[77]
	Parametric	Ranking	
Bi-objective knapsack	Dichotomic	Branch and bound	[102]
Bi-objective knapsack	Dichotomic	Ranking	[42]
Three-objective knapsack	Dichotomic	Ranking	[42]

1.5 Multi-objective Branch and Bound

Branch and bound is a standard method to solve single objective combinatorial optimisation problems and is contained in any textbook on combinatorial optimisation. In order to apply it to multi-objective combinatorial optimisation problems, we need to define the branching and the bounding parts of the algorithm. Branching refers to the method used to partition the feasible set of a combinatorial optimisation problem into two or more disjoint subsets that define new subproblems. Bounding refers to

the computation of lower and upper bounds (bound sets), see Section 1.1.4, and their use to eliminate subproblems from further consideration. Apart from branching and bounding, a strategy to select the next subproblem for evaluation is required. We address these questions based on contributions to the literature in turn.

1.5.1 Branching and Node Selection

The branching strategy concerns only the feasible set of the problem, hence there is no difference in partitioning the feasible set of a subproblem between the single and multiple objective cases. Nevertheless, the rules used to decide the branching may involve the objective function coefficients. Several authors propose branching strategies for the multi-objective binary knapsack problem

$$\max \left\{ \left(\sum_{j=1}^n c_j^1 x_j, \dots, \sum_{j=1}^n c_j^p x_j \right) : \sum_{j=1}^n w_j x_j \leq \omega, x \in \{0, 1\}^p \right\}, \quad (1.18)$$

where c_j^k and w_j are non-negative integer numbers. For $p = 1$, algorithms often use the rank of a variable according to decreasing value-to-weight ratio c_j/w_j . They define the branching strategy by setting variables with small rank equal to one and those with high rank equal to 0, see [45].

In the multi-objective case, [98] propose to consider the rank r_j^k of each item j according to ratio c_j^k/w_j for $k \in \{1, \dots, p\}$, and to order the items by decreasing value of the sum of the ranks $\sum_{k=1, \dots, p} r_j^k$. [3] use an order obtained from the decreasing sum of ratios $\sum_{k=1, \dots, p} c_j^k/w_j$, increasing worst rank $\max_{k=1, \dots, p} r_j^k$ and best rank $\min_{k=1, \dots, p} r_j^k$. [42] proposes orders respecting the dominance relation between vectors of ratios $(c_j^1/w_j, \dots, c_j^p/w_j)$. [42] also proposes to count for each item j the number of items $|dom(j)|$ dominating it, ordering items by increasing value of $|dom(j)|$. Finally, [42] suggests to order items according increasing rank following a principle from evolutionary multi-objective algorithms, see [19]. The choice of order has a considerable impact on solution times, as [42] demonstrates.

Other branching strategies that are popular in single objective branch and bound methods require the evaluation of (relaxed) solutions of subproblems (such as computing the fractionality of variables in the LP relaxation of a binary optimisation problem) and have not yet been modified for the use in multi-objective branch and bound algorithms.

Typical node selection strategies follow a depth-first or breadth-first principle and can be adopted to multi-objective branch and bound without changes. In fact, most published algorithms follow a depth-first strategy. More elaborate methods, such as best-first strategies on the other hand require adaptations such as the comparison of bound sets, and the difficulties this raises have so far prevented researchers to implement such strategies.

1.5.2 Bounding and Fathoming Nodes

Multi-objective branch and bound algorithms from the literature maintain a set of (feasible) points that defines an upper bound set U and the algorithms terminate as soon as $U = Y_N$. U can be initialised as $\{(\infty, \dots, \infty)\}$ or by a set of feasible solutions that are obtained by heuristic methods. The upper bound set is updated any time a new feasible solution is found. For each subproblem, or node of the branch and bound tree, a lower bound set L for the set Y_N of non-dominated points of the subproblem is computed.

A node of the branch and bound tree can be fathomed if

1. The subproblem has an empty feasible set (infeasibility);
2. The non-dominated set Y_N of the subproblem belongs to L (optimality);
3. For every $l \in L$ there exists some $u \in U$ such that $u \leq l$ (dominance).

Notice that fathoming a subproblem by optimality is often only practically possible if L consists of a single feasible point. The general condition $Y_N \subseteq L$ is, however, very difficult to verify. Indeed, because L is generally a continuous set, whereas U is a discrete set of points, fathoming a node by dominance (or even optimality) is not trivial. Several researchers have proposed methods for the practical application of the dominance test. [75] describe a method which, for a given (feasible) upper bound set U computes a set of points $D(U)$ such that the condition that the non-dominated set lies between the lower and upper bound set can be written in a way that is easier to use algorithmically, see Figures 1.11a and 1.11b for illustrations of $D(U)$.

Proposition 1. *Let L and U be upper bound sets for Y_N . Then*

$$(L + \mathbb{R}_{\leq}^p) \setminus (U + \mathbb{R}_{>}^p) = (L + \mathbb{R}_{\leq}^p) \cap \bigcup_{u \in D(U)} (u - \mathbb{R}_{\leq}^p). \quad (1.19)$$

Then, all points in L are weakly dominated by points in U if and only if all points in $D(U)$ are not dominated by any points in L . This rule for fathoming by dominance is illustrated in Figure 1.11a. In the case that all $y \in Y$ are integer vectors, the fathoming rule can be improved, as shown in Figure 1.11b, for details we have to refer to [91] and [75].

[91] reformulate the dominance condition via strictly monotone functions $h : \mathbb{R}^p \rightarrow \mathbb{R}$, i.e. $y^1 < y^2$ implies $h(y^1) < h(y^2)$. If a strictly monotone function h is such that $h(l) \geq 0$ for all $l \in L$ and $h(u) \leq 0$ for all $u \in U$ then $h(y) \geq 0$ for all $y \in Y_N$. [91] provide classes of functions h for the case that $L + \mathbb{R}_{\leq}^p$ is a polyhedron and $p = 2$.

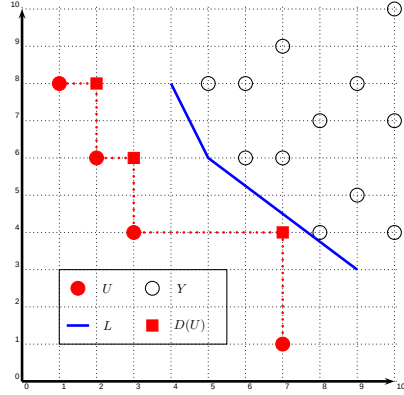


Fig. 1.11a The node can be fathomed by dominance

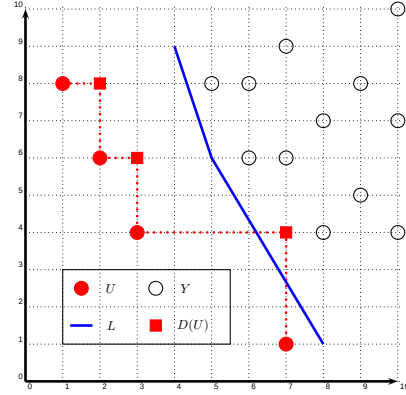


Fig. 1.11b The node can be fathomed by dominance assuming $Y \in \mathbb{Z}^p$

1.5.3 Multi-objective Branch and Bound Algorithms from the Literature

In this section, we provide a tabular summary of multi-objective branch and bound algorithm from the literature. For each algorithm in Table 1.5, we first mention the MOCO problem to which the algorithm is applied, where BO stands for bi-objective, TO for three-objective and MO for multi-objective. Then we provide specifications for the algorithm, namely lower and upper bound sets used, the branching and node selection strategies and the method used to find new feasible solutions. Finally, we list the reference. Note that the column headings refer to minimisation problems, so the interpretation of lower and upper bound set is reversed for problems that are formulated with maximisation objectives.

1.6 Conclusion

In this chapter we have summarised exact algorithmic approaches for solving multi-objective combinatorial optimisation problems. We have focused on general algorithmic schemes of increasing complexity. Starting from dynamic programming and greedy schemes, we also considered scalarisation and general algorithms based on scalarisation and the two-phase method as a specific technique for multi-objective optimisation. Finally, we covered multi-objective branch and bound algorithms. We do not claim that the surveys in any of these sections are exhaustive, since new algorithms appear all the time, and researchers in diverse fields are today involved in the development of new algorithms. A lot of algorithms are also being developed specifically for particular MOCO problems. We just mention [58] for an adapta-

Table 1.5 Multi-objective branch and bound algorithms

MOCO problem	Upper bound	Lower bound	Branching	Node selection	Feasible solutions	Reference
MO binary	Incumbent set	Utopia point	Variable fixing	Depth-first	Variable fixing	[47]
BO knapsack	Incumbent set	Utopia point	Variable fixing	Depth-first	Variable fixing	[98]
BO knapsack	Adaptation of [98] restricted to triangles					[102]
BO spanning tree	Incumbent set	Utopia point	Edge fixing	Depth-first	At leaves	[80]
BO spanning tree	Incumbent set	Convex relaxation	Edge fixing	Depth-first	Convex relaxation at nodes	[91]
MO knapsack	Incumbent set	Ideal point LP relaxation	Variable fixing	Depth-first	At leaves	[35]
TO knapsack	Incumbent set	Utopia point	Variable fixing	Depth-first	At leaves	[42]
TO knapsack	Incumbent set	Convex relaxation	Variable fixing	Lexicographic order of 5 criteria	Part of UB	[42]
BO assignment	Adaptation of [91] restricted to triangles					[20]
BO flow shop	No details	No details	No details	Depth-first	No details	[60]
BO mixed integer	Incumbent set	Ideal point of LP relaxation	Variable fixing	Depth-first	LP at leaves	[56, 57]
BO mixed integer	Extended incumbent set	Ideal point of LP relaxation	Variable fixing	Depth-first	LP at leaves	[101]

tion of the concept of the core of a knapsack problem to the bi-objective case and [9] for non-additive multi-objective shortest path problems. Furthermore, new general algorithmic ideas begin to make an appearance in multi-objective combinatorial optimisation. [43] combine an ε -constraint method with branch-and-cut to solve the bi-objective covering tour problem. Recently, researchers are developing algorithms that use (single objective) integer programming solvers as a black box. These approaches exploit the tremendous advantages that have been made in single objective combinatorial optimisation algorithms and solvers, see [10] for one such method. Also, new methodologies using tools from algebraic geometry emerge, see [7]. We have not touched at all on the vast field of heuristic and metaheuristic algorithms for MOCO problems, or the increasing interest that approximation algorithms received, see e.g. [4] and references therein. All of these established research directions and newly emerging areas offer plenty open questions for future research.

References

1. H. Aissi, R. Mahjoub, S. T. McCormick, and M. Queyranne. A strongly polynomial time algorithm for multicriteria global minimum cuts. In J. Lee and J. Vygen, editors, *Integer*

- Programming and Combinatorial Optimization. 17th International Conference, IPCO 2014, Bonn, Germany, June 23-25, 2014, Proceedings*, volume 8494 of *Lecture Notes in Computer Science*, pages 25–36. Springer, 2014.
2. Y.P. Aneja and K.P.K. Nair. Bicriteria transportation problem. *Management Science*, 25:73–78, 1979.
 3. C. Bazgan, H. Hugot, and D. Vanderpooten. Solving efficiently the 0-1 multi-objective knapsack problem. *Computers & Operations Research*, 36:260–279, 2009.
 4. C. Bazgan, F. Jamain, and D. Vanderpooten. Approximate Pareto sets of minimal size for multi-objective optimization problems. *Operations Research Letters*, 43:1–6, 2015.
 5. R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16:87–90, 1958.
 6. J.-F. Bérubé, F. Gendreau, and J.-Y. Potvin. An exact ϵ -constraint method for bi-objective combinatorial optimization problems: Application to the travelling salesman problem with profits. *European Journal of Operational Research*, 194:39–50, 2009.
 7. V. Blanco and J. Puerto. Partial Gröbner bases for multiobjective integer linear optimization. *SIAM Journal on Discrete Mathematics*, 23:571–595, 2009.
 8. V. Blanco and J. Puerto. A new complexity result on multiobjective linear integer programming using short rational generating functions. *Optimization Letters*, 6:537–543, 2012.
 9. L. Blander Reinhardt and D. Pisinger. Multi-objective and multi-constrained non-additive shortest path problems. *Computers & Operations Research*, 38:605–616, 2011.
 10. N. Boland, H. Charkhgard, and M. Savelsbergh. The triangle splitting method for biobjective mixed integer programming. In J. Lee and J. Vygen, editors, *Integer Programming and Combinatorial Optimization. 17th International Conference, IPCO 2014, Bonn, Germany, June 23-25, 2014, Proceedings*, volume 8494 of *Lecture Notes in Computer Science*, pages 162–173. Springer, 2014.
 11. P.M. Camerini, G. Galbiati, and F. Maffioli. The complexity of multi-constrained spanning tree problems. In L. Lovasz, editor, *Theory of Algorithms*, pages 53 – 101. North-Holland, 1984.
 12. L.G. Chalmet, L. Lemonidis, and D.J. Elzinga. An algorithm for the bi-criterion integer programming problem. *European Journal of Operational Research*, 25:292–300, 1986.
 13. C.R. Chegireddy and H.W. Hamacher. Algorithms for finding k -best perfect matchings. *Discrete Applied Mathematics*, 18:155–165, 1987.
 14. J.L. Cohon. *Multiobjective Programming and Planning*. Academic Press, New York, 1978.
 15. H.W. Corley. Efficient spanning trees. *Journal of Optimization Theory and Applications*, 45:481–485, 1985.
 16. K. Dächert, J. Gorski, and K. Klamroth. An augmented weighted tchebycheff method with adaptively chosen parameters for discrete bicriteria optimization problems. *Computers & Operations Research*, 39:2929–2943, 2012.
 17. K. Dächert and K. Klamroth. A linear bound on the number of scalarizations needed to solve discrete tricriteria optimization problems. *Journal of Global Optimization*, 61:643–676, 2015.
 18. H.G. Daellenbach and C.A. De Kluyver. Note on multiple objective dynamic programming. *Journal of the Operational Research Society*, 31:591–594, 1980.
 19. K. Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001.
 20. C. Delort. *Algorithmes d'énumération implicite pour l'optimisation multi-objectifs: Exploitation d'ensembles bornant et application aux problèmes de sac à dos et d'affectation*. PhD thesis, Université Pierre et Marie Curie (Paris VI), 2011.
 21. C. Delort and O. Spanjaard. Using bound sets in multiobjective optimization: Application to the biobjective binary knapsack problem. In P. Desta, editor, *Experimental Algorithms. 9th International Symposium, SEA 2010, Ischia Island, Naples, Italy, March 20-22, 2010, Proceedings*, volume 6049 of *Lecture Notes in Computer Science*, pages 253–265. Springer, 2010.
 22. R. Dial. A model and algorithm for multicriteria route-mode choice. *Transportation Research B*, 13:311–316, 1979.
 23. E.W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

24. M. Ehrgott. On matroids with multiple objectives. *Optimization*, 38:73–84, 1996.
25. M. Ehrgott. A discussion of scalarization techniques for multiobjective integer programming. *Annals of Operations Research*, 147:343–360, 2005.
26. M. Ehrgott. *Multicriteria Optimization*. Springer, 2nd edition, 2005.
27. M. Ehrgott and X. Gandibleux. Bounds and bound sets for biobjective combinatorial optimization problems. In M. Köksalan and St. Zionts, editors, *Multiple Criteria Decision Making in the New Millennium*, volume 507 of *Lectures Notes in Economics and Mathematical Systems*, pages 241–253. Springer, 2001.
28. M. Ehrgott and X. Gandibleux. Bound sets for biobjective combinatorial optimization problems. *Computers and Operations Research*, 34:2674–2694, 2007.
29. M. Ehrgott and K. Klamroth. Connectedness of efficient solutions in multiple criteria combinatorial optimization. *European Journal of Operational Research*, 97(1):159–166, 1997.
30. M. Ehrgott and D.M. Ryan. Constructing robust crew schedules with bicriteria optimization. *Journal of Multi-Criteria Decision Analysis*, 11:139–150, 2002.
31. M. Ehrgott and D.M. Ryan. The method of elastic constraints for multiobjective combinatorial optimization and its application in airline crew scheduling. In T. Tanino, T. Tanaka, and M. Inuiguchi, editors, *Multi-Objective Programming and Goal-Programming: Theory and Applications*, volume 21 of *Advances in Soft Computing*, pages 117–122. Springer, 2003.
32. M. Ehrgott and D. Tenfelde-Podehl. Computation of ideal and nadir values and implications for their use in MCDM methods. *European Journal of Operational Research*, 151:119–131, 2003.
33. V.A. Emelichev and V.A. Perepelitsa. On cardinality of the set of alternatives in discrete many-criterion problems. *Discrete Mathematics and Applications*, 2:461–471, 1992.
34. A. Eusébio and J.R. Figuiera. Finding non-dominated solutions in bi-objective integer network flow problems. *Computers & Operations Research*, 36:2554–2564, 2009.
35. K. Florios, G. Mavrotas, and D. Diakoulaki. Solving multiobjective, multiconstraint knapsack problems using mathematical programming and evolutionary algorithms. *European Journal of Operational Research*, 203:14–21, 2010.
36. K. Fukuda and T. Matsui. Finding all the perfect matchings in bipartite graphs. *Networks*, 22:461–468, 1992.
37. J. Gorski, L. Paquete, and F. Pedrosa. Greedy algorithms for a class of knapsack problems with binary weights. *Computers & Operations Research*, 39:498–511, 2012.
38. J. Gorski, L. Paquete, and F. Pedrosa. Greedy algorithms for a class of knapsack problems with binary weights. *Computers & Operations Research*, 39:498–511, 2012.
39. H.W. Hamacher and G. Ruhe. On spanning tree problems with multiple objectives. *Annals of Operations Research*, 52:209–230, 1994.
40. P. Hansen. Bicriterion path problems. In G. Fandel and T. Gal, editors, *Multiple Criteria Decision Making Theory and Application*, volume 177 of *Lecture Notes in Economics and Mathematical Systems*, pages 109–127. Springer, 1979.
41. M. Iori, S. Martello, and D. Pretolani. An aggregate label setting policy for the multi-objective shortest path problem. *European Journal of Operational Research*, 207:1489–1496, 2010.
42. J. Jorge. *Nouvelles propositions pour la résolution exacte du sac à dos multi-objectif unidimensionnel en variables binaires*. PhD thesis, Université de Nantes, 2010.
43. N. Jozefowiez, F. Semet, and E.-G. Talbi. The bi-objective covering tour problem. *Computers & Operations Research*, 34:1929–1942, 2007.
44. R.M. Karp. Reducibility among combinatorial problems. In R.E. Miller and J.W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
45. H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, 2004.
46. G. Kirlik and S. Sayin. A new algorithm for generating all nondominated solutions of multiobjective discrete optimization problems. *European Journal of Operational Research*, 232:479–488, 2014.
47. G. Kiziltan and E. Yucaoglu. An algorithm for multiobjective zero-one linear programming. *Management Science*, 29:1444–1453, 1983.

48. K. Klamroth, R. Lacour, and D. Vanderpooten. On the representation of the search region in multi-objective optimization. *European Journal of Operational Research*, 2015.
49. D. Klein and E. Hannan. An algorithm for the multiple objective integer linear programming problem. *European Journal of Operational Research*, 9:378–385, 1982.
50. P. Kouvelis and R.C. Carlson. Total unimodularity applications in bi-objective discrete optimization. *Operations Research Letters*, 11:61–65, 1992.
51. J.B. Kruskal. On the shortest spanning subtree of a graph and the travelling salesman problem. *Proceedings of the American Mathematical Society*, 7:48–50, 1956.
52. H.W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics*, 2:83–97, 1955.
53. M. Laumanns, L. Thiele, and Zitzler E. An efficient, adaptive parameter variation scheme for metaheuristics based on the epsilon-constraint method. *European Journal of Operational Research*, 169:932–942, 2006.
54. H. Lee and P.S. Pulat. Bicriteria network flow problems: Integer case. *European Journal of Operational Research*, 66:148–157, 1993.
55. M. Lokman, B. and Köksalan. Finding all nondominated points of multi-objective integer programs. *Journal of Global Optimization*, 57:347–365, 2013.
56. G. Mavrotas and D. Diakoulaki. A branch and bound algorithm for mixed zero-one multiple objective linear programming. *European Journal of Operational Research*, 107(3):530–541, 1998.
57. G. Mavrotas and D. Diakoulaki. Multi-criteria branch and bound: A vector maximization algorithm for mixed 0-1 multiple objective linear programming. *Applied Mathematics and computation*, 171(3):53–71, 2005.
58. G. Mavrotas, J. R. Figueira, and K. Florios. Solving the bi-objective multi-dimensional knapsack problem exploiting the concept of core. *Applied Mathematics and Computation*, 215:2502–2514, 2009.
59. K. Mavrotas, G. and Florios. An improved version of the augmented ϵ -constraint method (augmecon2) for finding the exact pareto set in multi-objective integer programming problems. *Applied Mathematics and Computation*, 219:9652–9669, 2013.
60. M. Mezma, N. Melab, and E.G. Talbi. A grid-based parallel approach of the multi-objective branch and bound. In *15th EUROMICRO International Conference on Parallel, Distributed and Network-Based Processing (PDP07)*, 2007.
61. J. Mote, I. Murthy, and D.L. Olson. A parametric approach to solving bicriterion shortest path problems. *European Journal of Operational Research*, 53:81–92, 1991.
62. M. Müller-Hannemann and K. Weihe. On the cardinality of the Pareto set in bicriteria shortest path problems. *Annals of Operations Research*, 147:269–286, 2006.
63. P. Neumayer. Complexity of optimization on vectorweighted graphs. In A. Bachem, U. Derigs, M. Jünger, and R. Schrader, editors, *Operations Research 93*, pages 359–361. Physica Verlag, Heidelberg, 1994.
64. M. Özlen and M. Azizoglu. Multi-objective integer programming: A general approach for generating all non-dominated solutions. *European Journal of Operational Research*, 199:25–35, 2009.
65. M. Ozlen, B.A. Burton, and C.A.G. MacRae. Multi-objective integer programming: An improved recursive algorithm. *Journal of Optimization Theory and Applications*, 160:470–482, 2014.
66. Ö. Özpeynirci and M. Köksalan. An exact algorithm for finding extreme supported nondominated points of multiobjective mixed integer programs. *Management Science*, 56:2302–2315, 2010.
67. J.M. Paixão and J.L. Santos. Labeling methods for the general case of the multi-objective shortest path problem – a computational study. In A. Madureira, C. Reis, and V. Marques, editors, *Computational Intelligence and Decision Making: Trends and Applications*, volume 61 of *Intelligent Systems, Control and Automation: Science and Engineering*, pages 489–502. Springer, 2013.
68. L. Paquete, M. Jaschob, K. Klamroth, and J. Gorksi. On a biobjective search problem in a line: Formulations and algorithms. *Theoretical Computer Science*, 507:61–71, 2013.

69. C. R. Pedersen, L. R. Nielsen, and K. A. Andersen. The bicriterion multi modal assignment problem: Introduction, analysis, and experimental results. *INFORMS Journal on Computing*, 20:400–411, 2008.
70. J.C. Prim. Shortest connection networks and some generalizations. *Bell System Technics Journal*, 36:1389–1401, 1957.
71. A. Przybylski, X. Gandibleux, and M. Ehrgott. The biobjective integer minimum cost flow problem – Incorrectness of Sedeño-Noda and González-Martín’s algorithm. *Computers & Operations Research*, 33:1459–1463, 2006.
72. A. Przybylski, X. Gandibleux, and M. Ehrgott. Computational results for four exact methods to solve the three-objective assignment problem. In V. Barichard, M. Ehrgott, X. Gandibleux, and V. T’Kindt, editors, *Multiple Objective Programming and Goal Programming: Theoretical Results and Practical Applications*, volume 618 of *Lecture Notes in Economics and Mathematical Systems*, pages 79–88. Springer, 2008.
73. A. Przybylski, X. Gandibleux, and M. Ehrgott. Two phase algorithms for the bi-objective assignment problem. *European Journal of Operational Research*, 185:509–533, 2008.
74. A. Przybylski, X. Gandibleux, and M. Ehrgott. A recursive algorithms for finding all non-dominated extreme points in the outcome set of a multiobjective integer program. *INFORMS Journal on Computing*, 22:371–386, 2010.
75. A. Przybylski, X. Gandibleux, and M. Ehrgott. A two phase method for multi-objective integer programming and its application to the assignment problem with three objectives. *Discrete Optimization*, 7:149–165, 2010.
76. A. Przybylski, X. Gandibleux, and M. Ehrgott. The two-phase method for multiobjective combinatorial optimization problem. In R. Majoub, editor, *Progress in Combinatorial Optimization*, pages 559–596. ISTE Wiley, 2011.
77. A. Raith and M. Ehrgott. A comparison of solution strategies for biobjective shortest path problems. *Computers & Operations Research*, 36:1299–1331, 2009.
78. A. Raith and M. Ehrgott. A two-phase algorithm for the biobjective integer minimum cost flow problem. *Computers & Operations Research*, 36:1945–1954, 2009.
79. T.K. Ralphs, M.J. Saltzmann, and M.M. Wiecek. An improved algorithm for solving biobjective integer programs. *Annals of Operations Research*, 147:43–70, 2006.
80. R.M. Ramos, S. Alonso, J. Sicilia, and C. González. The problem of the optimal biobjective spanning tree. *European Journal of Operational Research*, 111:617–628, 1998.
81. A. Rong and J.R. Figueira. Computational performance of basic state reduction based dynamic programming algorithms for bi-objective 0-1 knapsack problems. *Computers and Mathematics with Applications*, 63:1462–1480, 2012.
82. G. Ruhe. Complexity results for multicriteria and parametric network flows using a pathological graph of Zadeh. *Zeitschrift für Operations Research*, 32:9–27, 1988.
83. S. Ruzika. *On Multiple Objective Combinatorial Optimization*. PhD thesis, Department of Mathematics, Technical University of Kaiserslautern, 2007.
84. S. Ruzika and H. W. Hamacher. A survey on multiple objective minimum spanning tree problems. In D. Lerner, D. Wagner, and K. A. Zweig, editors, *Algorithmics*, volume 5515 of *Lecture Notes in Computer Science*, pages 104–116. Springer, 2009.
85. P. Sanders and L. Mandow. Parallel label-setting multi-objective shortest path search. In *2013 IEEE 27th International Symposium on Parallel & Distributed Processing (IPDPS)*, pages 215–224. IEEE, 2013.
86. S. Sayin and P. Kouvelis. The multiobjective discrete optimization problem: A weighted min-max two-stage optimization approach and a bicriteria algorithm. *Management Science*, 51:1572–1581, 2005.
87. A. Sedeño-Noda and C. González-Martín. An algorithm for the biobjective integer minimum cost flow problem. *Computers & Operations Research*, 28:139–156, 2001.
88. A. Sedeño-Noda and C. González-Martín. An efficient label setting/correcting shortest path algorithm. *Computational Optimization and Applications*, 51:437–455, 2012.
89. A. Sedeño-Noda and A. Raith. A dijkstra-like method computing all extreme supported non-dominated solutions of the biobjective sortest path problem. *Computers & Operations Research*, 57:83–94, 2015.

90. P. Serafini. Some considerations about computational complexity for multi objective combinatorial problems. In J. Jahn and W. Krabs, editors, *Recent Advances and Historical Development of Vector Optimization*, volume 294 of *Lecture Notes in Economics and Mathematical Systems*, pages 222–232. Springer, 1986.
91. F. Sourd and O. Spanjaard. A multiobjective branch-and-bound framework: Application to the biobjective spanning tree problem. *INFORMS Journal on Computing*, 20:472–484, 2008.
92. M. Stanojević, M. Vujosevic, and B. Stanojević. On the cardinality of the nondominated set of multi-objective combinatorial optimization problems. *Operations Research Letters*, 41:197–200, 2013.
93. S. Steiner and T. Radzik. Computing all efficient solutions of the biobjective minimum spanning tree problem. *Computers & Operations Research*, 35:198–211, 2008.
94. J. Sylva and Crema A. A method for finding the set of non-dominated vectors for multiple objective integer linear programs. *European Journal of Operational Research*, 158:46–55, 2004.
95. B. Tam, D. Ryan, and M. Ehrgott. Multi-objective approaches to the unit crewing problem in airline crew scheduling. *Journal of Multi-criteria Decision Analysis*, 21:257–277, 2014.
96. Z. Tarapata. Selected multicriteria shortest path problems: An analysis of complexity, models and adaptation of standard algorithms. *International Journal of Applied Mathematics and Computer Science*, 1:269–287, 2007.
97. D. Tuytens, J. Teghem, P. Fortemps, and K. Van Nieuwenhuyse. Performance of the MOSA method for the bicriteria assignment problem. *Journal of Heuristics*, 6:295–310, 2000.
98. E. Ulungu and J. Teghem. Solving multi-objective knapsack problem by a branch-and-bound procedure. In J. Climaco, editor, *Multicriteria Analysis*, pages 269–278. Springer, 1997.
99. E.L. Ulungu and J. Teghem. The two phases method: An efficient procedure to solve bi-objective combinatorial optimization problems. *Foundations of Computing and Decision Sciences*, 20:149–165, 1995.
100. B. Villarreal and M.H. Karwan. Multicriteria integer programming: A (hybrid) dynamic programming recursive approach. *Mathematical Programming*, 21:204–223, 1981.
101. T. Vincent, F. Seipp, S. Ruzika, A. Przybylski, and X. Gandibleux. Multiple objective branch and bound for mixed 0-1 linear programming: Corrections and improvements for the biobjective case. *Computers & Operations Research*, 40:498–509, 2013.
102. M. Visée, J. Teghem, M. Pirlot, and E.L. Ulungu. Two-phases method and branch and bound procedures to solve the bi-objective knapsack problem. *Journal of Global Optimization*, 12:139–155, 1998.
103. D.J.A. Welsh. *Complexity: Knots, Colourings and Counting*. Cambridge University Press, 1993.
104. A.P. Wierzbicki. On the completeness and constructiveness of parametric characterizations to vector optimization problems. *OR Spektrum*, 8:73–87, 1986.